

Theoretical Foundation of ZK-SNARK Video Steganography

VideoLevel technical design note

19 September 2026 (revised 5 October 2026)

Abstract

This design note formalizes the construction currently implemented by VideoLevel. A message m and a compressed Groth16 proof π are packed into a payload, wrapped in a plain length-prefixed frame (channel protocol v3, no MAC), whitened with an HMAC keystream and written into the first trailing-one sign bit of selected CAVLC residual blocks of H.264 IDR slices. Positions are chosen by a keyed pseudorandom schedule, so extraction is blind: it needs neither the cover video nor a sidecar. The proof states that *a camera registered under a public Poseidon Merkle root vouches for exactly this video and this message*; the verifier needs the root, the verification key and the stego key, but no camera secret, and does not learn which camera produced the proof. The note states each security property together with the assumption it rests on, and lists the properties the construction does not provide.

Update 05/10/2026. This revision replaces the 03/10/2026 description in two places. (i) *Channel protocol v3* (2026-10-04): the frame no longer carries the 16-byte HMAC tag of protocol v2 and the subkey K_f is gone; the frame is authenticated only through the Groth16 proof. HKDF labels, whitening and the per-IDR schedule are unchanged. (ii) *Camera proof*: the former circuit `payload_verify.circom` (63 321 constraints, 513 public signals, statement $c = \text{SHA256}(\text{SHA256}(m) \parallel K)$, so the verifier had to hold the same secret) has been replaced by `camera_video.circom` (8 737 constraints, 3 public inputs): registry membership of a camera key plus a binding to a digest of the video and to the message. The former Python pipeline (Python CAVLC parser, position safety filter, chaos maps, `positions.json` sidecars, strict mode that re-parsed the cover, the SEC1–SEC10 suite) was removed on 2026-10-03 and is *not* part of the system described here. Streams produced by protocol v1 or v2 cannot be read by v3, and proofs made with the previous circuit keys do not verify.

1 Notation and trust boundary

Bytes are big-endian and bit strings are MSB-first. $uk(x)$ denotes the k -bit big-endian encoding of an integer x , $X[a:b]$ the byte slice $a, \dots, b-1$, and $\text{HMAC}(k, x)$ always means HMAC-SHA256. Let V be a Baseline-profile H.264 Annex-B elementary stream and m a non-empty byte string. Two unrelated secrets are involved:

- the *stego key* $K \in \{0, 1\}^{256}$, shared by embedder and verifier; it selects carrier positions and whitens the frame;
- the *camera secret* $s \in \mathbb{F}_r$ (BN254 scalar field), held only by the camera; the public registry stores $\text{pk} = \text{Poseidon}(s)$.

Let $\beta \geq 1$ be the per-segment bit budget (`max_bits_per_IDR`, default 64) and $L_{\max}$ the configured maximum payload size (4096 bytes in the native CLI; the frame format allows up to $2^{16} - 1$). Both are public parameters that embedder and receiver must agree on.

The embedder outputs a stego stream V' only; no sidecar is produced. The verifier holds V' , K , β , $L_{\max}$, the Groth16 verification key vk and a trusted registry root ρ ; it never holds s . Neither K , s nor m may be written to job status records, logs or benchmark artifacts.

2 Carrier: trailing-one sign bits

CAVLC codes each 4×4 (or DC) residual block as the syntax element `coeff_token` (the pair (TotalCoeff, TrailingOnes), VLC table chosen by nC), one `trailing_ones_sign_flag` per trailing one, the remaining levels, `total_zeros` and `run_before`.

Definition 1 (Candidate). Let $\sigma = (\text{SPS}, \text{PPS}, \text{IDR})$ be a segment. For every residual block of category $t \in \{0, 1, 2, 3\}$ (LumaDC, Luma4x4, ChromaDC, ChromaAC) in the IDR slice with `TrailingOnes` ≥ 1 , the candidate is the *first* `trailing_ones_sign_flag` of that block in bitstream order. It is identified by

$$\text{id}(c) = \text{ASCII}(n : a : t : b : o),$$

where n is the NAL index inside the segment, a the macroblock address, b the block index, and o the RBSP bit offset of the sign flag. $C(\sigma)$ denotes the set of candidates of σ ; at most one candidate exists per block.

Property 1 (Length and context preservation). Flipping a candidate sign changes one coefficient from ± 1 to ∓ 1 and leaves TotalCoeff, TrailingOnes, every codeword length and every nC value unchanged. Hence the RBSP length, all RBSP bit offsets, the parse of every later syntax element, and $C(\sigma)$ itself are identical in cover and stego. (Emulation-prevention bytes are re-inserted when RBSP is converted back to EBP, so EBP byte offsets may shift; identities and the video digest use RBSP.)

Property 1 follows from the CAVLC syntax and is checked bit-exactly by the implementation; it is what makes blind extraction possible: the receiver recomputes $C(\sigma)$ from V' . A flip still changes the decoded picture (a dequantized coefficient change of magnitude 2 at quantized level, propagated through intra prediction and deblocking); imperceptibility is an empirical property measured by decoded PSNR/SSIM, not a theorem. Only IDR slices carry bits; non-IDR NAL units are copied unchanged.

3 Key schedule (protocol v3)

Definition 2 (Subkeys). With HKDF-SHA256 (RFC 5869),

$$\begin{aligned} \text{PRK} &= \text{HMAC}(\text{"zkstego-cavlc-v2-salt"}, K), \\ K_s &= \text{HKDF-Expand}(\text{PRK}, \text{"zkstego/cavlc/v2/schedule"}, 32), \\ K_w &= \text{HKDF-Expand}(\text{PRK}, \text{"zkstego/cavlc/v2/whitening"}, 32). \end{aligned}$$

The labels keep the v2 string of the previous protocol, so `schedule` and `whitening` are bit-identical to v2; only the frame changed. HMAC serves here purely as a keyed pseudorandom function, not as an authenticator.

Definition 3 (Segment schedule). Let V' be split into segments $\sigma_0, \sigma_1, \dots$, one per IDR NAL, each consisting of the latest SPS and PPS followed by that IDR. In segment σ_j , order the candidates by the key $(\text{HMAC}(K_s, \text{id}(c)), \text{id}(c))$ ascending (lexicographically on bytes) and keep the first

$$q_j = \min(\beta, |C(\sigma_j)|)$$

candidates. Concatenating these lists over $j = 0, 1, \dots$ gives the global position sequence $p_0, p_1, \dots$; frame bit i is written at p_i . Bit indices continue across segments; scheduled positions beyond the frame length keep their cover sign. Embedding fails if $\sum_j q_j < |F|$, where $|F|$ is the frame length in bits.

The identity contains no sign value and $C(\sigma_j)$ is sign-invariant (Property 1), so a receiver with K and β reproduces $p_0, p_1, \dots$ exactly from V' . A mismatched β yields a different schedule and extraction fails. We write $\Gamma(V, K, \beta, |F|) = \{p_0, \dots, p_{|F|-1}\}$ for the *carrier set*.

4 Whitened frame

Definition 4 (Frame and embedded bits). For a payload P with $|P| \leq L_{\max}$,

$$F = 0x03 \parallel u16(|P|) \parallel P.$$

The whitening keystream is $S = \text{HMAC}(K_w, u64(0)) \parallel \text{HMAC}(K_w, u64(1)) \parallel \dots$ and the embedded bit at p_i is

$$e_i = F_i \oplus S_i, \quad \text{sign}(p_i) = \begin{cases} + & e_i = 0 \\ - & e_i = 1. \end{cases}$$

The frame overhead is 3 bytes, so $|F| = 8(|P| + 3)$ bits. Protocol v2 also appended the tag $\text{HMAC}(K_f, \cdot)[0:16]$; removing it saves 128 bits (two IDRs at $\beta = 64$) because the mandatory Groth16 check already rejects any modified message or proof.

Blind extraction. (1) Parse the IDR segments of V' and enumerate $C(\sigma_j)$. (2) Derive K_s, K_w and the schedule. (3) Read and de-whiten the first 24 bits; reject unless the version is `0x03` and the `u16` length is $\leq L_{\max}$. (4) Read and de-whiten the remaining $8|P|$ bits; reject if the stream ends early. The output P is an *unauthenticated candidate*: the extract command and the extract job return it with `"verified": false`. With a wrong key the de-whitened header is essentially random, so a wrong key passes step (3) with probability about $2^{-8} \cdot (L_{\max} + 1)/2^{16} \approx 2.4 \cdot 10^{-4}$ for $L_{\max} = 4096$; the remaining cases fail proof decompression or Groth16 verification.

5 Video digest and binding

Definition 5 (Video digest). Let $\phi = |F|$ (known before proving, since $|P|$ depends only on $|m|$). For every NAL unit u of V in order, let h_u be its header byte and r'_u its RBSP (emulation-prevention bytes removed) in which exactly the bits at positions of $\Gamma(V, K, \beta, \phi)$ are cleared to 0. Then

$$D(V) = \text{SHA256}\left(\text{"zkstego/video-digest/v1"} \parallel u32(\phi) \parallel u32(\beta) \parallel \big\|_u (u32(1 + |r'_u|) \parallel h_u \parallel r'_u)\right).$$

Property 2 (Digest invariance). $D(V') = D(V)$ for a stego stream V' produced from cover V with K and β : embedding changes only bits in Γ , Γ is identical for V and V' (Property 1), and EPB changes do not affect RBSP. The implementation (`zkstego_blind_bits video-digest`, reference `src/video_binding.py`) checks `native = Python` and `cover = stego`.

Definition 6 (Binding). For mode $\mu \in \{0, 1\}$ and digest D (with $D = 0^{256}$ when $\mu = 1$),

$$b = \text{SHA256}(\text{"zkstego/proof-binding/v1"} \parallel u8(\mu) \parallel D \parallel \text{SHA256}(m)),$$

split as $b_{\text{hi}} = b[0:16]$ and $b_{\text{lo}} = b[16:32]$, read as two 128-bit integers, which fit in $\mathbb{F}_r$ without reduction. Mode 0 binds the video; mode 1 (message only) exists for live streams, whose remaining frames do not exist when the proof is embedded.

6 Payload, registry and zero-knowledge relation

Definition 7 (Payload). $P = 0x01 \parallel u8(\mu) \parallel u16(|m|) \parallel m \parallel \pi$, where π is a Groth16 proof over BN254 compressed to 129 bytes (the x -coordinates of $A \in G_1$, $B \in G_2$ (two $\mathbb{F}_p$ elements), $C \in G_1$, 32 bytes each, plus one byte holding the three y -parity bits).

The length field gives an unambiguous message/proof split. With $L_{\max} = 4096$ the largest message is $4096 - 4 - 129 = 3963$ bytes. Decompression rejects coordinates $\geq p$ and points not on the curve.

Definition 8 (Registry). The registry is a binary Merkle tree of depth 16 (at most 65 536 cameras) with nodes $\text{Poseidon}(\ell, r)$, empty leaves 0 and leaf i equal to $\text{pk}_i = \text{Poseidon}(s_i)$ (`src/camera_registry.py`; Poseidon with the circomlib constants, cross-checked against circomlibjs vectors and the circuit witness). Its root ρ is public; the verifier trusts only ρ .

Definition 9 (Relation). The circuit `camera_video.circom` (8 737 constraints, public inputs $\rho, b_{\text{hi}}, b_{\text{lo}}$) defines

$$\mathcal{R} = \left\{ (\rho, b_{\text{hi}}, b_{\text{lo}}; s, \mathbf{z}, \mathbf{d}) : \text{MerkleRoot}(\text{Poseidon}(s), \mathbf{z}, \mathbf{d}) = \rho, d_i \in \{0, 1\} \right\},$$

where $\mathbf{z} = (z_0, \dots, z_{15})$ are the siblings and $\mathbf{d} = (d_0, \dots, d_{15})$ the path indices; at level i the running node and z_i are ordered by d_i and hashed with $\text{Poseidon}(2)$.

Booleanity $d_i(d_i - 1) = 0$ is necessary for soundness: without it the selection $\ell = x + d(z - x)$ could mix the two children and open non-members. $b_{\text{hi}}, b_{\text{lo}}$ appear in no other constraint; they are squared inside the circuit so that they are explicitly part of the R1CS (as in Semaphore). In Groth16 every public input enters the verification equation, so a proof is tied to the exact triple $(\rho, b_{\text{hi}}, b_{\text{lo}})$.

Verification. The verifier accepts (m, π) for a received stream V' if and only if all of the following hold: blind extraction returns a frame; P unpacks with format byte `0x01`, $\mu \in \{0, 1\}$ and a valid 129-byte encoding; $\mu = 0$ (mode 1 is rejected unless `ZK_STEGO_ALLOW_MESSAGE_ONLY_PROOFS=1`); and

$$\text{Groth16.Verify}(\text{vk}, (\rho, b_{\text{hi}}, b_{\text{lo}}), \pi) = 1,$$

where ρ is the trusted root and b is recomputed by the verifier from $D(V')$ (with $\phi = 8(|P| + 3)$) and m ; public signals supplied by the sender are never used. `snarkjs` must exit with code 0 and print `OK!`. The HTTP verify job reports `succeeded` with m and `video_bound` only in that case; otherwise `rejected` with reason `payload_not_found`, `malformed_proof_payload`, `video_binding_required`, `video_digest_unavailable` or `proof_invalid`, and failed on infrastructure errors (no registry configured, missing key, Node.js unavailable).

7 Security properties

Assumption 1 (Standard primitives). (A1) HMAC-SHA256 is a pseudorandom function (PRF) when keyed with a uniformly random 32-byte key; consequently HKDF-SHA256 is a secure KDF and K_s, K_w are computationally independent pseudorandom keys when K is uniform. (A2) SHA-256 is collision resistant. (A3) Poseidon over BN254 is collision and preimage resistant. (A4) Groth16 is knowledge-sound and zero-knowledge in the generic-group/algebraic model, and a proof for one statement cannot be turned into a proof for a different statement without a witness (Groth16 proofs are only re-randomizable for the same public inputs), *provided* the setup was generated honestly and its trapdoor destroyed.

The following properties hold under Assumption 1; they are standard reductions and no stronger claim is made.

Property 3 (Pseudorandom schedule). For a party without K , the ranking of candidates in every segment is computationally indistinguishable from a uniformly random permutation chosen independently of the cover (by A1 applied to K_s , given distinct identities). Hence the selected positions are pseudorandom to such a party.

Property 4 (No fixed-header bias). For a single frame per key, S is indistinguishable from uniform by A1 applied to K_w , so the embedded bits e_i are pseudorandom even though F has a fixed version byte, small length field and structured proof bytes. Selected signs therefore carry no fixed header pattern.

Property 5 (Subkey separation). Disclosure of one subkey (e.g. K_s) reveals neither K nor the other subkey, by A1 applied to PRK and the HKDF-Expand outputs.

Property 6 (Registered source). If Verify accepts, then by A4 the prover knew $(s, \mathbf{z}, \mathbf{d})$ with $\text{MerkleRoot}(\text{Poseidon}(s), \mathbf{z}, \mathbf{d}) = \rho$, and by A3 $\text{Poseidon}(s)$ is a leaf of the trusted registry, i.e. the prover holds the secret of a registered camera. A camera outside the registry (even with a valid proof for its own tree) is rejected because its root differs from ρ .

Property 7 (Video and message binding). Let π verify for (ρ, b) with b derived from $(D(V'), m)$. For another stream V'' or message m'' with $D(V'') \neq D(V')$ or $m'' \neq m$, the recomputed binding differs except with negligible probability (A2), so by A4 π does not verify for it and a new proof needs a registered secret. This covers replaying the payload into another video, truncation, dropping or adding frames, editing a later frame, flipping a non-carrier sign and re-encoding. Flipping a carrier bit leaves D unchanged (Property 2) but changes the extracted frame, hence m or π , and is rejected as well.

Property 8 (Camera privacy). By the zero-knowledge property of A4, π reveals nothing about s or the leaf index beyond membership in the tree with root ρ ; the verifier does not learn which registered camera produced the proof, and never needs s .

These outcomes are exercised by an 11-case attack matrix (benchmark run of 2026-10-04, all as expected): the untouched stego is accepted; wrong stego key, a dropped carrier IDR and a re-encode find no frame (`payload_not_found`); a flipped scheduled sign gives a malformed proof; a flipped non-carrier sign in a carrier IDR, a flipped sign in a later frame, truncation to half, a swapped message, a camera outside the registry and the payload replayed into another video all fail Groth16 (`proof_invalid`).

Properties not provided

- **Live streams bind only the message.** A live stream embeds its proof before the rest of the video exists, so it uses mode 1 ($D = 0^{256}$); such a payload could be replayed into another stream under the same K . The verify job therefore rejects mode 1 unless explicitly allowed.
- **Verification needs the stego key.** No camera secret is needed, but K is required to extract and to locate the carrier bits that D clears; the digest is therefore key-dependent and verification is not public for parties without K . A holder of K can extract and re-embed but cannot forge a proof for a new video or message without a registered s .
- **No channel integrity by itself.** Since v3 the frame has no MAC; an extracted payload is only a candidate until the proof verifies.
- **No nonce in whitening.** S depends only on K . Embedding two frames under the same K reuses the keystream; the guarantee above is per single frame per key. Positions remain hidden, which limits but does not remove this leakage. Whitening is not an IND-CPA encryption of m .
- **No undetectability proof.** Pseudorandom positions and bits do not imply that the sign changes are statistically undetectable relative to natural cover statistics; detectability must be measured (ML steganalysis has not been measured).

- **Trusted setup.** Phase 1 is the public Hermez Powers of Tau (2^{16} , BLAKE2b hash pinned); Phase 2 consists of two contributions, a random beacon generated locally and `snarkjs zkey verify`, all run on one machine. This is a demo ceremony: whoever controlled that machine could know the trapdoor and forge proofs, so A4 does not hold in a deployment sense until independent parties contribute (one honest contributor suffices).
- **No proof of a real scene.** The proof shows that a registered camera key vouched for the bytes, not that the sensor captured a real scene (e.g. re-filming a screen), nor that the key is protected by hardware.
- **Robustness.** The channel is fragile by design: any re-encode, transcode or bit error in a selected sign destroys the frame or the proof.

8 Algorithms

Embedding (`src.camera_proof.build_payload` + native embed)

1. Validate $|K| = 32$, $0 < |m|$, $4 + |m| + 129 \leq L_{\max}$; load circuit artifacts and the registry path of the camera.
2. Compute $\phi = 8(4 + |m| + 129 + 3)$, the digest $D(V)$ of the cover with K and β , and the binding b for mode 0.
3. Run Groth16 with private $(s, \mathbf{z}, \mathbf{d})$ and public $(\rho, b_{\text{hi}}, b_{\text{lo}})$; compress π to 129 bytes; form P .
4. Derive K_s, K_w ; form F ; for each IDR segment in order, schedule q_j positions and set $\text{sign}(p_i)$ from e_i until all $|F|$ bits are placed.
5. Re-insert emulation prevention and emit V' ; other NAL units are copied.

Verification (`src.camera_proof.verify_payload`, verify job)

1. Blind extraction (Section 4) with $K, \beta, L_{\max}$; reject on version, length or capacity failure.
2. Unpack P , decompress π , reject malformed encodings before pairing checks; reject mode 1 unless allowed.
3. Recompute $D(V')$ and b (Section 5) and run Groth16 verification against the trusted ρ ; accept only on success.

9 Capacity and evaluation quantities

The capacity of a stream is $\sum_j \min(\beta, |C(\sigma_j)|)$ bits and a message of $|m|$ bytes needs $|F| = 8(|m| + 4 + 129 + 3) = 8(|m| + 136)$ bits. In the recorded benchmark run (2026-10-04) a 68-byte message gives a 201-byte payload and $|F| = 1632$ bits, which occupy $\lceil 1632/64 \rceil = 26$ IDRs at $\beta = 64$; about half of the scheduled signs actually flip. The 300-frame all-intra CIF clip used for end-to-end timing has 188 556 candidates and 19 200 bits of capacity at $\beta = 64$.

Evaluations should report candidate counts, scheduled bits, decoded quality, verification outcome and latency per stage separately; measured values are artifact-specific, not guarantees. The 2026-10-04 run (9 clips $\times$ 3 resolutions, QP 22, all-intra Baseline CAVLC, 30 frames) passed 27/27 media cases with video-bound proofs and no sign changed outside the schedule; the lowest cover-to-stego PSNR-Y per case was 44.03–58.68 dB (0/27 below 40 dB); the share of negative trailing ones moved by $|z| \leq 0.22$; Groth16 proving took 1.5–1.8 s and verification 0.9–1.0 s for the 129-byte proof (PLONK on the same R1CS: about 22 s to prove). Native parsing, not Groth16,

bounds throughput: the digest parses every carrier IDR. The strongest regime is all-intra Baseline CAVLC; with $\text{GOP} > 1$, P-frames referencing modified IDR frames inherit the distortion and need separate measurement.

10 Operational surface

The HTTP service exposes public `/health` and bearer-token-protected `embed`, `extract`, `verify`, `status` and one-time artifact download jobs plus a WebSocket stream endpoint. The token (`ZK_STEGO_API_TOKEN`, at least 32 characters) is checked before the body is read; repeated authentication failures are rate-limited; uploads, concurrency and job queues are bounded. The `verify` job needs the trusted registry (`ZK_STEGO_CAMERA_REGISTRY`) and fails without it; `extract` results and live payloads carry `"verified": false`; job states are `queued`, `running`, `succeeded`, `rejected` (proof not verified, never success) and `failed`. Status records never contain K , s , m or stack traces. Native processes receive keys on stdin and are run without a shell and with timeouts.

The signed-manifest and expiring-certificate modules (Ed25519) remain in the repository as auxiliary trust interfaces but are not part of the embedding or verification path above. An offline verifier can reject an expired certificate, but cannot prevent use of a copied key with a modified clock; hard expiry needs an online lease, trusted time or hardware-enforced key.

11 Limitations and target design

The native parser accepts only Baseline CAVLC, progressive 4:2:0, one slice per frame starting at macroblock 0, no I_PCM, and a single SPS/PPS id; only IDR frames carry data. Realtime operation is not accepted: the corrected physical-camera run (2026-09-24, protocol v1, before the 2026-10-02 native speed-up) reached 7.608 source FPS against a 30-FPS gate and has not been re-measured.

`future_plan.md` describes the remaining target design, which is **not implemented**: per-segment commitments and session chaining with a final seal so that live streams bind more than the message; hardware-backed capture attestation; a multi-party Groth16 ceremony; and a public, sign-independent carrier schedule so that verification no longer needs the stego key. None of these properties should be attributed to the current construction.